



Arquitectura Frontend + Integración de Datos

Documento técnico completo — Septiembre 2026

Este documento describe la arquitectura del frontend de VYGO, los flujos de datos desde el agente de IA hacia la interfaz del repartidor, la integración con Supabase y el modelo de datos que conecta la base de datos con los componentes visuales.

React 18 + TypeScript

Supabase Auth + DB

MapLibre GL JS

Zustand

Tailwind CSS v3

Vercel

1. Stack técnico

1.1 Tecnologías principales

Capa	Tecnología	Versión	Propósito
UI	React	18	Renderizado declarativo
Lenguaje	TypeScript	5	Tipado estático
Build	Vite	5.4	Dev server + bundler
Estilos	Tailwind CSS	v3	Utility-first, tokens de diseño

Routing	React Router	v6	SPA navigation + guards
Estado global	Zustand	5	Stores reactivos con <code>persist</code>
Servidor HTTP	TanStack Query	v5	Cache de queries remotas
Mapas	MapLibre GL JS	v6	Mapa vectorial en tiempo real
Tiles	MapTiler <code>dataviz</code>	—	Estilo claro con identidad de marca
Auth + DB	Supabase	2.x	Auth OAuth + PostgreSQL + RLS
Deploy	Vercel	—	Producción: <code>vygo-ten.vercel.app</code>

1.2 Variables de entorno

```
VITE_SUPABASE_URL=https://ihmadvmoenkxanoxrwcj.supabase.co
VITE_SUPABASE_ANON_KEY=<anon key>
VITE_MAPTILER_KEY=<maptiler key>
```

2. Estructura de archivos

```
src/
├─ App.tsx           ← Router principal + guards RequireAuth/RequireGuest
├─ main.tsx          ← Bootstrap, Supabase onAuthStateChanged
├─ index.css         ← Tailwind base + estilos globales + MapLibre overrides
├─ lib/
│   ├─ supabase.ts   ← Cliente Supabase singleton
│   ├─ utils.ts       ← formatCurrency, formatDistance, formatMinutes, cn()
│   └─ vygoAgent.ts  ← Tipos TypeScript del contrato del agente (PDF oficial)
├─ types/
│   ├─ order.ts       ← Order, OrderStatus, Platform
│   ├─ driver.ts      ← Driver, DriverStatus, DriverState
│   ├─ route.ts       ← RouteStop, RouteState
│   └─ earnings.ts    ← EarningsSummary, HourlyEarning, PlatformEarning
├─ stores/
│   └─ auth.store.ts  ← Estado global Zustand
│                       ← isAuthenticated, userId, email, phone
```

└─ driver.store.ts	← status, todayEarnings, earningsPerHour, completedOrders
└─ orders.store.ts	← activeOrders, completedOrders, pendingOffer
└─ route.store.ts	← currentStopIndex, stops, totalStops
└─ hooks/	
└─ useNewOrder.ts	← Timer que simula/recibe nuevas ofertas; vibración
└─ useActiveRoute.ts	← currentStop, nextStop, totalStops
└─ useEndShift.ts	← tryEndShift, confirmEnd – con guard de pedidos activos
└─ useCountUp.ts	← Animación numérica (ease-out cubic)
└─ useLocationTracking.ts	← GPS → Supabase ubicaciones_conductores cada 8s
└─ usePlatformConnections.ts	← CRUD de platform_connections en Supabase
└─ services/	
└─ mock-data.ts	← Pedidos mock, driver mock, earnings mock (Monterrey)
└─ orders.service.ts	← getActiveOrders(), generateNewOffer()
└─ agent.service.ts	← decidir() con cascade: agente → B2 → static
└─ earnings.service.ts	← getSummary(period) – datos de ganancias
└─ components/	
└─ layout/	
└─ AppLayout.tsx	← Shell principal: mobile full-width / desktop phone frame
└─ AuthLayout.tsx	← Shell auth: mobile full-width / desktop phone frame
└─ BottomNavigation.tsx	← Nav fija; badge en "Pedidos" con activeOrders.length
└─ PageHeader.tsx	← Header reutilizable con título y back button
└─ maps/	
└─ MockMap.tsx	← MapLibre GL JS: GPS real, markers, ruta, followDriver
└─ orders/	
└─ NewOrderSheet.tsx	← Bottom sheet principal: countdown 30s, agente IA, swipe
└─ ui/	← Button, Input, Badge, Tabs, Card, Progress
└─ PlatformBadge.tsx	← Badge visual por plataforma (Uber/Rappi/DiDi)
└─ PlatformConnectModal.tsx	← Modal conexión de plataformas → Supabase
└─ DriverStatusBadge.tsx	← Badge de estado del repartidor
└─ EndShiftConfirm.tsx	← Confirmación de fin de jornada
└─ pages/	
└─ Login/index.tsx	← Google OAuth + email/password
└─ Register/index.tsx	← Registro con email/password
└─ Home/index.tsx	← Dashboard principal (offline/online)
└─ ActiveRoute/index.tsx	← Mapa a pantalla completa + overlays
└─ Orders/index.tsx	← Tabs: En curso / Completados
└─ OrderDetail/index.tsx	← Detalle de un pedido
└─ Earnings/index.tsx	← Ganancias con datos reales del store
└─ Profile/index.tsx	← Perfil + plataformas + logout

3. Routing y guards de autenticación

```
// App.tsx – estructura de rutas

<RequireGuest>           ← redirige a "/" si ya hay sesión
  /login                  → LoginPage
  /register                → RegisterPage

<RequireAuth>             ← redirige a "/login" si no hay sesión
  /                        → HomePage
  /orders                 → OrdersPage
  /orders/:id             → OrderDetailPage
  /route                   → ActiveRoutePage
  /earnings                → EarningsPage
  /profile                 → ProfilePage
```

RequireAuth lee `useAuthStore(s => s.isAuthenticated)`. La sesión se restaura en `main.tsx` mediante `supabase.auth.getSession()` antes de montar React. El listener `onAuthStateChange` captura `SIGNED_IN`, `TOKEN_REFRESHED` y `SIGNED_OUT` en tiempo real.

Importante: Si Supabase no devuelve sesión válida en `initSession()`, el store limpia `isAuthenticated=false` eliminando cualquier estado persistido inválido. Esto resuelve el error 401 por sesiones expiradas.

4. Autenticación con Supabase

4.1 Métodos disponibles

Método	Implementación	Estado
Google OAuth	<code>signInWithOAuth({ provider: 'google' })</code>	✅ Activo

Apple OAuth	<code>signInWithOAuth({ provider: 'apple' })</code>	🕒 Pendiente config
Email + contraseña	<code>signInWithPassword({ email, password })</code>	✅ Activo
Registro email	<code>signUp({ email, password, data: { full_name } })</code>	✅ Activo
Phone OTP	<code>signInWithOtp({ phone })</code> + <code>verifyOtp()</code>	🕒 Requiere Twilio

4.2 Flujo OAuth (Google)

```

Usuario toca "Continuar con Google"
  |
  ▼
signInWithOAuth({ provider: 'google', redirectTo: origin + '/' })
  | ← redirige al proveedor
  ▼
Google devuelve token → Supabase callback
  |
  ▼
onAuthStateChange(SIGNED_IN, session)
  |
  ▼
useAuthStore.setState({ isAuthenticated: true, userId, email })
  |
  ▼
RequireGuest detecta sesión → navega a "/"
  
```

4.3 Seguridad de sesión

- El `access_token` de Supabase expira en **1 hora** por defecto.
- El refresh es automático via el cliente Supabase JS.
- `TOKEN_REFRESHED` es capturado en `onAuthStateChange` y actualiza el store.
- Al cerrar sesión: `supabase.auth.signOut()` + limpieza del store + navegación a `/login`.

5. Base de datos Supabase

5.1 Tablas del backend de negocios

Estas tablas fueron diseñadas para el backend del sistema (ver documento [vygo-ai-training.pdf](#) para detalle completo):

Tabla	Filas actuales	Propósito
<code>apps</code>	3	Catálogo: uber, rappi, didi
<code>usuarios</code>	6	Clientes + repartidores
<code>repartidores</code>	3	Perfil operativo
<code>pedidos</code>	2	Órdenes de entrega
<code>viajes_repartidor</code>	3	Viajes activos con ruta PostGIS
<code>viaje_pedidos</code>	0	Pivote viaje ↔ pedido
<code>ofertas_pedido</code>	0	Historial de ofertas
<code>difusiones_pedido</code>	0	Rondas de búsqueda
<code>configuracion</code>	7	Parámetros del sistema

5.2 Tablas del frontend (nuevas)

PLATFORM_CONNECTIONS

Almacena qué plataformas tiene conectadas cada repartidor en la app.

Columna	Tipo	Descripción
<code>id</code>	uuid PK	Identificador
<code>user_id</code>	uuid FK → auth.users	Dueño del registro
<code>platform</code>	text CHECK	<code>'uber'</code> <code>'rappi'</code> <code>'didi'</code>

<code>driver_alias</code>	text nullable	Alias del repartidor en esa plataforma
<code>city</code>	text nullable	Ciudad de operación
<code>connected_at</code>	timestampz	Cuándo se conectó
<code>is_active</code>	boolean	Soft-delete (disconnect = false)

RLS: `FOR ALL TO authenticated USING (auth.uid() = user_id)`

Constraint UNIQUE: `(user_id, platform)` — una fila por plataforma por usuario.

UBICACIONES_CONDUCTORES

Actualizada por el hook `useLocationTracking` cada 8 segundos o 15 metros.

Columna	Tipo	Descripción
<code>user_id</code>	uuid PK	FK → auth.users
<code>lat</code>	float8	Latitud actual
<code>lng</code>	float8	Longitud actual
<code>updated_at</code>	timestampz	Última actualización

Operación: `upsert` con `onConflict: 'user_id'` — siempre una sola fila por conductor.

6. Stores Zustand

6.1 auth.store.ts

```
interface AuthStore {
  isAuthenticated: boolean
  email: string | null
  phone: string | null
  userId: string | null
}
```

```
// Métodos:
login(email, password) // email + contraseña
register(name, email, pass) // registro nuevo usuario
signInWithGoogle() // OAuth → redirect
signInWithApple() // OAuth → redirect
signInWithPhone(phone) // SMS OTP paso 1
verifyPhoneOtp(phone, token) // SMS OTP paso 2
logout()
initSession() // restaurar sesión al cargar
}
```

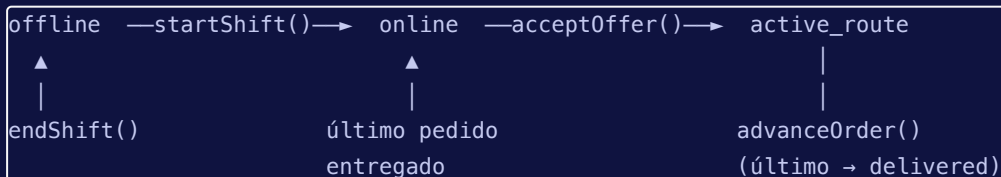
Persistencia: `zustand/middleware persist` con key `'vygo-auth'`. Se limpia automáticamente si `initSession()` no encuentra sesión válida en Supabase.

6.2 driver.store.ts

```
interface DriverStore {
  driver: Driver // info: name, vehicle, rating, platforms
  status: DriverStatus // 'offline' | 'online' | 'active_route'
  todayEarnings: number // actualizado por addEarnings() al entregar
  earningsPerHour: number // recalculado = todayEarnings / horasTrabajadas
  completedOrders: number // incrementado por incrementCompletedOrders()
  shiftStartedAt: Date | null

  startShift() // status → 'online', guarda shiftStartedAt
  endShift() // status → 'offline', limpia shiftStartedAt
  setStatus(status) // transición directa
  addEarnings(amount) // += amount; recalcula earningsPerHour
  incrementCompletedOrders()
}
```

Flujo de status del repartidor:



6.3 orders.store.ts

```
interface OrdersStore {
  activeOrders: Order[] // pedidos en ruta actual
  completedOrders: Order[] // historial del turno
  pendingOffer: Order | null // oferta esperando decisión
  currentEarningsPerHour: number // $/h proyectado con la ruta actual
  _nextRouteNumber: number // counter estable (no se renumera)

  acceptOffer(order) // → activeOrders, status='active_route'
  rejectOffer() // limpia pendingOffer
  setPendingOffer(order) // coloca oferta para mostrar en NewOrderSheet
  advanceOrder(id) // heading_to_pickup → picked_up → delivered
  updateOrderStatus(id, status)
  simulateNewOrder() // genera oferta mock para demos
  resetShift() // limpia activeOrders, pendingOffer, _nextRouteNumber=1
}
```

Regla de routeNumber : se asigna al aceptar la oferta y **nunca cambia**, aunque otros pedidos sean entregados antes. Garantiza estabilidad visual en el mapa.

6.4 route.store.ts

```
interface RouteStore {
  stops: RouteStop[] // lista plana de paradas [pickup1, dropoff1, pickup2...]
  currentStopIndex: number // índice de la parada activa
  advanceStop() // ++currentStopIndex
}
```

7. Flujo de una oferta — end to end

Este es el flujo más importante del sistema: desde que llega una oferta hasta que el repartidor la entrega.

```

[Backend / Simulador]
  |
  | POST /decidir (o simulateNewOrder() en mock)
  ▼
useNewOrder (hook)
  | setPendingOffer(order)
  | navigator.vibrate([200, 100, 200]) ← haptic feedback
  |
  ▼
NewOrderSheet se monta (pendingOffer != null)
  | Countdown: 30s → auto-rejectOffer() si llega a 0
  | decidir(order, rhoActual) → agent.service.ts
  |   | POST /decidir (2s timeout)
  |   | fallback: B2_umbral (tasa_marginal > rho_actual)
  |   | fallback: static replay_12.json
  | Muestra: earnings, distancia, ETA, explicación del agente
  | Usuario decide:
  |   | ACEPTAR:
  |   |   | acceptOffer(order)
  |   |   | status → 'active_route'
  |   |   | BottomNavigation badge++
  |   | RECHAZAR (botón / swipe 75px / countdown):
  |   |   | rejectOffer()
  |   |   | pendingOffer = null
  |   |
  |   ▼
[Si aceptó] ActiveRoute
  | MockMap: markers de pickup/dropoff + ruta
  | advanceOrder(id): heading_to_pickup → picked_up → delivered
  | Al picked_up: marker de recogida desaparece
  | Al delivered (último pedido):
  |   | addEarnings(order.earnings)
  |   | incrementCompletedOrders()
  |   | status → 'online'
  |   | navega a dashboard de jornada

```

8. Interpretación del contrato del agente IA

El agente devuelve un objeto `RespuestaDecidir` definido en `src/lib/vygoAgent.ts`. Aquí se describe cómo cada campo se mapea a la UI.

8.1 Tipos del contrato

```
// src/lib/vygoAgent.ts – tipos oficiales

interface DecisionOferta {
  aceptar: boolean // decisión principal
  explicacion_corta: string // 1 línea para el repartidor
  tasa_marginal: number // $/h si acepta este pedido
  rho_actual: number // $/h actual del turno
  ajuste_aprendido: number // bonus/malus por posición, opción, riesgo
  politica: 'B1' | 'B2' | 'PP0' | 'HIBRIDO'
}

interface Economia {
  tarifa: number // pesos que paga la plataforma
  propina_esperada: number // estimado estadístico
  costo_km: number // combustible + desgaste
  costo_tiempo: number // minutos * costo_τ
  ganancia_neta: number // tarifa + propina - costos
}

interface Riesgo {
  prob_retraso: number // 0-1: probabilidad de llegar tarde
  fresqueza_restante: number // minutos antes de que la comida se enfríe
  holgura: number // σ_i: minutos esperando en la cocina
  desvio_km: number // kilómetros extra vs ruta actual
}

interface Plan {
  secuencia: string[] // IDs de pedidos en orden óptimo
  tiempo_total_min: number
  distancia_total_km: number
  eta_ultimo: string // ISO timestamp del último dropoff
}
```

8.2 Mapeo agente → UI

Campo del agente	Dónde se muestra	Componente
------------------	------------------	------------

<code>aceptar</code>	Color del encabezado (verde/rojo) + icono	<code>NewOrderSheet</code>
<code>explicacion_corta</code>	Texto de 1 línea debajo del encabezado	<code>NewOrderSheet</code>
<code>tasa_marginal</code>	<code>\$X/h</code> en verde grande	<code>NewOrderSheet</code>
<code>rho_actual</code>	tu promedio: <code>\$Y/h</code> en gris	<code>NewOrderSheet</code>
<code>ajuste_aprendido</code>	Tercera métrica: <code>±\$Z/h ajuste</code>	<code>NewOrderSheet</code>
<code>politica</code>	Badge pequeño: <code>B2 / PPO / HÍBRIDO</code>	<code>NewOrderSheet</code>
<code>riesgo.prob_retraso</code>	Barra de riesgo (verde→rojo)	<code>NewOrderSheet</code>
<code>riesgo.fresqueza_restante</code>	Indicador "Frescura: Xmin"	<code>NewOrderSheet</code>
<code>riesgo.holgura</code>	Indicador "Holgura: Xmin"	<code>NewOrderSheet</code>
<code>economia.ganancia_neta</code>	Importe principal en bold	<code>NewOrderSheet</code>
<code>plan.secuencia</code>	Orden de marcadores en el mapa	<code>MockMap</code>
<code>plan.tiempo_total_min</code>	ETA en el banner de <code>ActiveRoute</code>	<code>ActiveRoute</code>

8.3 Cascade de fuentes del agente

```
// src/services/agent.service.ts

async function decidir(offer, rhoActual, posicion?) {
  // 1. Intenta el agente real (2s timeout)
  try {
    const resp = await POST('/decidir', payload, { timeout: 2000 })
    return { ...resp, _source: 'agent' }
  } catch {}

  // 2. Fallback B2: regla de umbral analítica
  //   Aceptar si tasa_marginal > rho_actual
  const tasa_marginal = calcularTasaMarginal(offer, rhoActual)
  if (tasa_marginal !== null) {
    return { aceptar: tasa_marginal > rhoActual, _source: 'b2_fallback', ... }
  }

  // 3. Fallback static: replay_12.json (para pitch)
  return { ...STATIC_REPLAY, _source: 'static' }
}
```

`_source` se muestra como badge pequeño en `NewOrderSheet` :

- `agent` →  Agente
- `b2_fallback` →  B2
- `static` →  Demo

9. Mapa en tiempo real (MockMap)

9.1 Inicialización

```
// Tile URL: MapTiler dataviz (estilo claro, referencia visual del brand)
const tiles = ['https://api.maptiler.com/maps/dataviz/{z}/{x}/{y}.png?key=${KEY}`']

// Posición inicial: localStorage primero, GPS inmediato al montar
const initPos = getSavedPosition() // clave: 'vygo-last-position'
getCurrentPosition(applyPosition) // centra el mapa al instante
watchPosition(applyPosition)      // actualizaciones continuas
```

9.2 Marcadores

Tipo	Forma	Color	Condición de visibilidad
Pickup #1	Círculo	Borde lima #6FA800	Solo si <code>status != 'picked_up'</code>
Pickup extra	Triángulo	Borde lima #6FA800	Solo si <code>status != 'picked_up'</code>
Dropoff	Óvalo	Naranja oscuro #C87000	Siempre
Driver	Punto pulsante	Lima #6FA800	Siempre cuando hay jornada

9.3 Ruta

La ruta se dibuja como `LineString` GeoJSON en MapLibre:

- **Casing (capa inferior):** blanco, ancho 8px, opacidad 50% — efecto de borde
- **Línea principal:** lima `#6FA800` , ancho 4px

9.4 followDriver

La prop `followDriver={true}` activa `map.easeTo({ center: lngLat })` cada vez que el GPS actualiza la posición. Se usa en `ActiveRoutePage` para modo conducción. Se implementa via `useRef` para evitar closures obsoletos.

10. Diseño visual — Tokens de color

10.1 Paleta

Token	Valor hex	Uso
<code>vygo-bg</code>	<code>#FAFBFF</code>	Fondo principal (blanco azulado)
<code>vygo-card</code>	<code>#F0F2FF</code>	Fondo de cards
<code>vygo-card-2</code>	<code>#E5E8FF</code>	Cards secundarios, inputs
<code>vygo-green</code>	<code>#6FA800</code>	Acento principal (lima oscuro)
<code>vygo-green-bright</code>	<code>#8CC800</code>	Hover de botones
<code>vygo-white</code>	<code>#0F1340</code>	Texto primario (navy oscuro)
<code>vygo-secondary</code>	<code>#4A5490</code>	Texto secundario (indigo)
<code>vygo-border</code>	<code>#C5CAF0</code>	Bordes sutiles
<code>vygo-warning</code>	<code>#C87000</code>	Advertencias, dropoff markers
<code>vygo-danger</code>	<code>#C42D2D</code>	Errores, acciones destructivas

10.2 Inspiración visual

La paleta proviene del logo oficial de VYGO:

- **Lima / verde-amarillo** (`#C9E86E` en logo, `#6FA800` oscurecido para legibilidad sobre blanco)
- **Índigo oscuro** (`#4B51A3` en logo → `#0F1340` para texto, `#0E1145` para fondo desktop)

10.3 Vista desktop (presentación)

En pantallas $\geq 1024\text{px}$, el app se renderiza dentro de un **marco de teléfono CSS**:

- Fondo: índigo oscuro `#0E1145` con blobs de luz lima/verde
- Marco: borde `#1a1f6e` , `border-radius: 52px` , shadow con halo lima
- Dynamic island y botones laterales en CSS puro
- Logo **VYGO** (blanco + lima) en esquina superior izquierda
- En mobile: sin marco, ocupa 100% de pantalla

11. Pantallas principales

11.1 Home — Estado offline

Muestra:

- Mapa compacto (180px) con overlay "Tu zona · Monterrey"
- Plataformas conectadas (desde `usePlatformConnections` → Supabase, datos reales)
- Stats de referencia: ganado hoy / entregas / promedio \$/h
- CTA "Comenzar jornada"

Acción: `startShift()` → status `'online'` → `shiftStartedAt = now()`

11.2 Home — Estado online

Muestra:

- Mapa grande (360px) con GPS en tiempo real
- HUD con ganancias animadas (useCountUp), pedidos, rendimiento
- Si hay pedidos activos: card "Siguiente entrega" con dirección y monto
- Si no hay pedidos: radar animado "Buscando el mejor pedido"
- Botón "Terminar jornada" (con guard: no permite si hay pedidos activos)

11.3 NewOrderSheet — La pantalla más importante

Aparece como bottom sheet sobre cualquier pantalla cuando `pendingOffer != null`.

Elementos UI:

1. **Barra de countdown** (30s): verde → naranja → rojo; auto-rechaza al llegar a 0
2. **Plataforma + número de pedido** + badge de política del agente
3. **Explicación corta** del agente (1 línea)
4. **Monto principal** en grande (earnings)
5. **Tres métricas del agente**: tasa_marginal / rho_actual / ajuste_aprendido
6. **Barras de riesgo**: probabilidad de retraso, frescura, holgura
7. **Info de ruta**: distancia, tiempo estimado, dirección de pickup
8. **Botones**: Rechazar (outline rojo) | Aceptar (líma)

Interacciones:

- Swipe down > 75px → rechazar
- Flash verde al aceptar (600ms) antes de ejecutar `acceptOffer()`
- Vibración al aparecer: `navigator.vibrate([200, 100, 200])`

11.4 ActiveRoute — Modo conducción

Pantalla especial sin navegación inferior, mapa a pantalla completa.

- **Banner superior flotante**: dirección actual + ETA en verde
- **Leyenda inferior izquierda**: tipos de marcadores
- **Card inferior**: barra de progreso + plataforma + botón de acción + siguiente pedido
- **Estado sin pedidos**: dashboard de jornada con proyecciones de ganancias

11.5 Ganancias

- Tab "Hoy": datos **reales** del store (`todayEarnings` , `earningsPerHour` , `completedOrders`)
- Si hay pedidos completados reales: breakdown por plataforma calculado dinámicamente
- Tab "Semana" / "Mes": datos mock de referencia
- Sección "Gracias a VYGO": ganancia adicional estimada, km ahorrados, tiempo ahorrado

11.6 Perfil — Plataformas conectadas

- Toca "Plataformas conectadas" → abre `PlatformConnectModal`
- El modal lee/escribe en Supabase `platform_connections`
- Conectar: pide alias opcional → upsert en DB
- Desconectar: soft-delete (`is_active = false`)
- Los badges del Home se actualizan en tiempo real vía `usePlatformConnections`

12. GPS y tracking en tiempo real

12.1 Mapa (MockMap)

```
// Inmediato al montar: centra el mapa
navigator.geolocation.getCurrentPosition(applyPosition, {
  enableHighAccuracy: true, timeout: 8000
})

// Continuo mientras la app está abierta
navigator.geolocation.watchPosition(applyPosition, {
  enableHighAccuracy: true, maximumAge: 3000
})
```

La posición se guarda en `localStorage['vygo-last-position']` para persistir entre recargas.

12.2 Supabase (useLocationTracking)

Se activa solo cuando `driverStatus !== 'offline'` y hay `userId`:

```
// Upsert cada 8s 0 cada 15m de movimiento (lo que ocurra primero)
await supabase
  .from('ubicaciones_conductores')
  .upsert({ user_id, lat, lng, updated_at: now() }, { onConflict: 'user_id' })
```

13. Conexión de plataformas — flujo completo

```
[Profile] usuario toca "Plataformas conectadas"
  |
  v
PlatformConnectModal se abre
  | usePlatformConnections() lee de Supabase
  | SELECT * FROM platform_connections WHERE user_id = auth.uid() AND is_active = true
  |
  | [Plataforma no conectada]
  |   Usuario toca "Conectar" → formulario con alias (opcional)
  |   UPSERT platform_connections { user_id, platform, driver_alias }
  |   Estado local actualizado optimísticamente
  |
  | [Plataforma conectada]
  |   Usuario toca "Desconectar"
  |   UPDATE platform_connections SET is_active = false
  |   Estado local actualizado
  |
  v
[Home] usePlatformConnections() reactivo → badges actualizados
```

RLS garantiza que cada usuario solo ve y modifica sus propias filas.

14. Datos mock para demos

El archivo `src/services/mock-data.ts` contiene:

PEDIDOS ACTIVOS (3 EN RUTA)

#	Plataforma	Monto	Distancia	ETA
1	Rappi #1842	\$71	1.8km	6min
2	Uber #8392	\$94	4.2km	14min
3	DiDi #7741	\$82	6.1km	19min

NUEVA OFERTA DEMO

Campo	Valor
Plataforma	Uber Eats
Monto	\$94
Tiempo extra	+8min
Distancia extra	+2.4km
Tasa proyectada	\$214/h

REPLAY ESTÁTICO (`PUBLIC/REPLAY_12.JSON`)

Para el pitch de IA — muestra 3 decisiones del agente:

- B1 (acepta todo): **\$612/turno**
- B2 (umbral): **\$948/turno**
- PPO (híbrido): **\$1,014/turno**

15. Estado de implementación

✅ Completo y funcional

- Auth completa con Google + email/contraseña
- Flujo de jornada: offline → online → active_route → offline
- NewOrderSheet con countdown, agente IA, swipe, vibración
- Mapa real con GPS, markers, ruta, followDriver
- Tracking GPS → Supabase en tiempo real
- Plataformas conectadas → Supabase con RLS
- Ganancias con datos reales del store
- Vista desktop con marco de teléfono CSS
- Diseño light theme: fondo blanco, acento lima, texto navy

🕒 Pendiente / configuración externa

Feature	Bloqueador
Apple Sign In	Config en Apple Developer + Supabase
Phone OTP	Proveedor SMS (Twilio) en Supabase
Pedidos reales desde Supabase	Conectar tabla <code>pedidos</code> + RLS para repartidor
Auth → perfil auto-creado	Trigger en <code>auth.users</code> → <code>usuarios</code>
Agente IA en producción	Endpoint <code>/decidir</code> en backend

✏️ Solo en desarrollo

- Botón "Simular pedido" (visible cuando jornada activa, `driverStatus !== 'offline'`)
- Mock data hardcodeada en `driver.service.ts` (earnings, completedOrders iniciales)

16. Deploy

Repositorio: `github.com/VYGO-Bug404/VYGO` **Producción:** `https://vygo-ten.vercel.app` **Framework detectado:** Vite (React SPA) **Build command:** `npm run build` **Output dir:** `dist/`

Las variables de entorno `VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY` y `VITE_MAPTILER_KEY` están configuradas en Vercel project settings.

VYGO v0.1 — Documento generado el 12 de septiembre de 2026